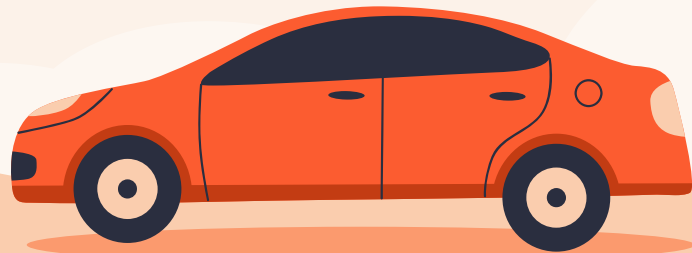


DriveShare

Maya Noureddine



DriveShare Design Pattern Project

For each design pattern I will discuss:

UML Diagram	UML class diagram of design pattern.
Implementation (code)	Source code showing the main parts.
WHAT/WHY	Why we chose this design pattern. What this feature does/how it does it.
Feature	The specific user stories of that feature.
Demo	I will demonstrate all the features of DriveShare, along with their respective design pattern if applicable.

GITHUB URL:



<https://github.com/mayanour4/DriveShare.git>

Video URL



<https://youtu.be/5sd8b1slZoo>

Table of contents

01

Singleton

02

Observer

03

Mediator

04

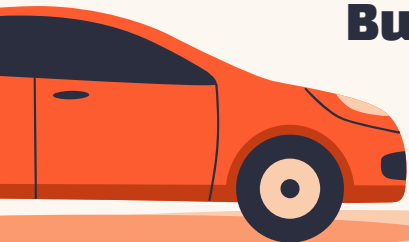
Builder

05

Proxy

06

**Chain of
Responsibility**



An illustration featuring two hands, one light orange and one brown, exchanging a dark blue car key. In the center, a white square with an orange border contains the number '01' in orange. The background is a light beige color with stylized white clouds, small orange stars, and a landscape at the bottom with orange hills and two stylized trees. The entire scene is framed by orange borders on the left and right sides.

01

Singleton Pattern

UML CLASS DIAGRAM

SessionManager

- currentUserId : int
- currentEmail : QString
- loggedIn : bool

+ getInstance() : SessionManager*
+ login(int, QString) : void
+ logout() : void
+ getUserId() : int
+ getEmail() : QString
+ isLoggedIn() : bool



Implementation

```
SessionManager* SessionManager::getInstance()  
{  
    static SessionManager instance;  
    return &instance;  
}
```

This creates the one and only object. Reused forever.

```
SessionManager.h  
  
1  #ifndef SESSIONMANAGER_H  
2  #define SESSIONMANAGER_H  
3  
4  #include <QString>  
5  
6  class SessionManager  
7  {  
8  public:  
9      static SessionManager* getInstance();  
10  
11     void login(int userId, const QString& email);  
12     void logout();  
13  
14     int getUserId() const;  
15     QString getEmail() const;  
16     bool isLoggedIn() const;  
17  
18 private:  
19     SessionManager();  
20  
21     int currentUserId;  
22     QString currentEmail;  
23     bool loggedIn;  
24 };  
25  
26 #endif // SESSIONMANAGER_H
```

Provides global access to the single instance

Private constructor



Feature: User Authentication & Registration

Session Manager WHY:

- Ensures only one user session exists throughout the entire application
- Prevents multiple users from being “logged in” at the same time within the same app instance

Session Manager WHAT:

- Stores the currently logged-in user’s information (User ID, Email, and Login status)
- Controls login and logout operations
- Allows the system to check if a user is authenticated

Register:

- Users can register on DriveShare using email and password authentication
- During registration, users must set three security questions (with answers).
- Users will be notified with a success message once registration is complete

DriveShare Login

Email:

Password:

[Forgot Password](#)

[Login](#) [Register](#)

Security Questions and Answers

Security question 1

Answer 1

Security question 2

Answer 2

Security question 3

Answer 3

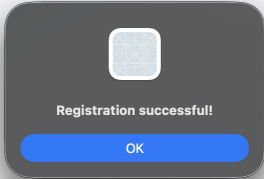
DriveShare Login

Email:

Password:

[Forgot Password](#)

[Login](#) [Register](#)



Security Questions and Answers

What was my first car?

What is my favorite color?

What is my favorite food?

Register:

- Users cannot register with the same email twice. No duplicate accounts allowed.
- User must fill out all fields when registering. Users must set three security questions and answers

DriveShare Login 

Email:

Password:

[Forgot Password](#)

[Login](#) [Register](#)

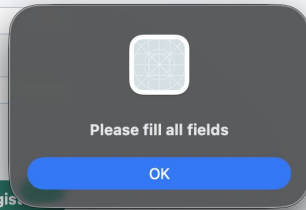
Security Questions and Answers 

What was my first car?

What is my favorite color?

Security question 3

Answer 3



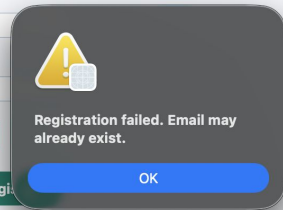
DriveShare Login 

Email:

Password:

[Forgot Password](#)

[Login](#) [Register](#)



LOG IN

Successful login:

- After successful registration, user can log into their account.
- User will be prompted with success message.

Failed login:

- User will not be able to login with incorrect username or email.
- UI will display Invalid login message

DriveShare Login

Email:

Password:

[Forgot Password](#)[Login](#)[Register](#)

Login successful!

[OK](#)

DriveShare Login

Email:

Password:

[Forgot Password](#)[Login](#)[Register](#)

Invalid login

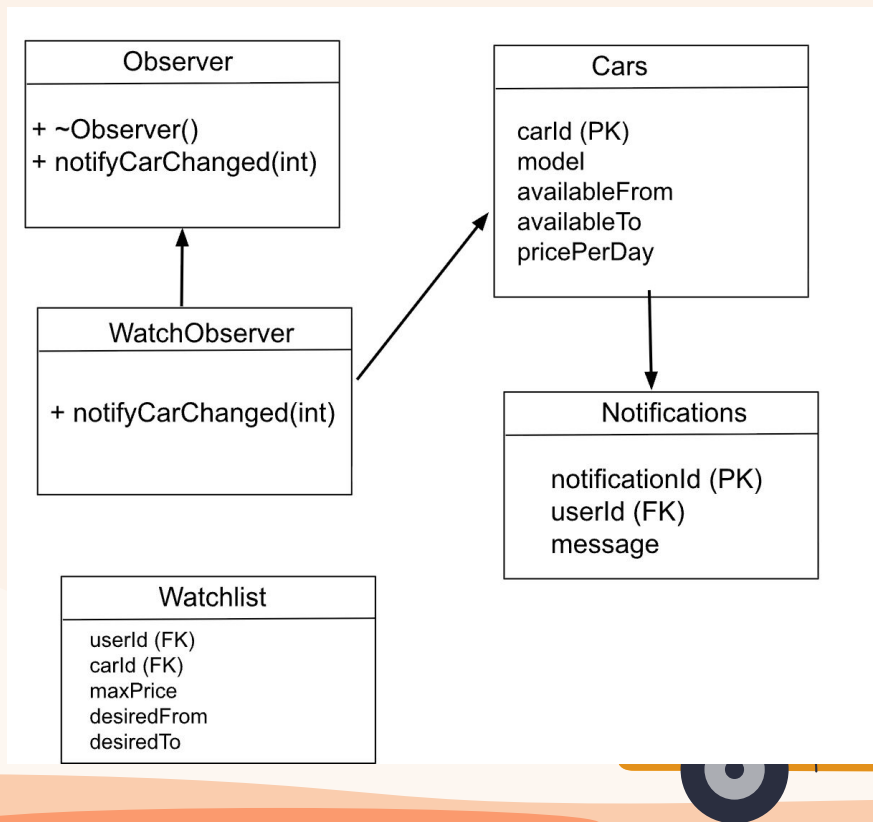
[OK](#)



02

Observer Pattern

UML CLASS DIAGRAM:



Implementation:

```
void WatchObserver::notifyCarChanged(int carId)
{
    // Load the car that changed or was added.
    QSqlQuery carQuery;
    carQuery.prepare(
        "SELECT model, availableFrom, availableTo, pricePerDay "
        "FROM Cars WHERE id = :carId"
    );
    carQuery.bindValue(":carId", carId);

    if (!carQuery.exec() || !carQuery.next()) {
        return;
    }
}
```

Notification logic

```
#ifndef OBSERVER_H
#define OBSERVER_H

// Observer: watches car changes and notifies matching renters.
class Observer
{
public:
    virtual ~Observer();
    // Called when a car is added or changed.
    virtual void notifyCarChanged(int carId) = 0;
};

// Checks the watchlist and creates notifications.
class WatchObserver : public Observer
{
public:
    void notifyCarChanged(int carId) override;
};

#endif // OBSERVER_H
```

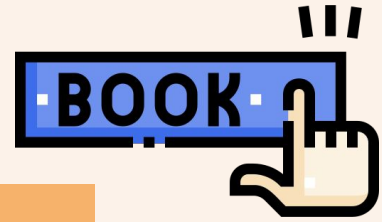
Observer
interface

Concrete
observer

The Observer pattern is implemented through the Observer interface and WatchObserver class, which automatically notify renters when watched cars become available or meet their specified price and date requirements.



Feature: Booking Notifications



Observer Pattern WHY:

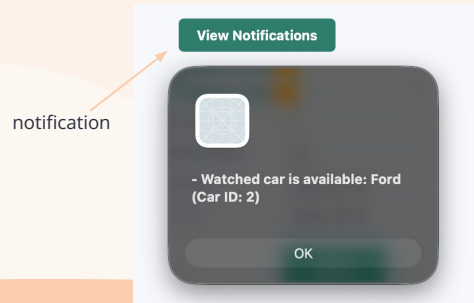
- Automatically informs renters about relevant car updates
- Eliminates the need for renters to repeatedly search for cars
- Decouples notification logic from car listing management

Observer Pattern WHAT:

- Observer defines the notification interface
- WatchObserver listens for car changes
- Matching watchlist users receive automatic notifications

Feature: Updating and Watching

- **Update Car Listing:** Owners have the ability to manage their car listings, including updating availability and price
- **Watching:** Renters can “watch” desired cars and receive notification when the car becomes available and/or rental prices drop to meet renter-defined



Watch a Car

Car ID:

Max Price:

From:

To:



Car added to watchlist!

Update Listing

Car ID:

From:

To:

Price/day:



Car listing updated successfully

Feature:

- **Booking:** Implement a booking system allowing renters to reserve a car for a specific period.
- **Not allowed:** User who listed the car, cannot book their own car
- **Double Renting:** The system prevents the same car from being rented by more than one renter for overlapping dates/times.

Book a Car ✓

Car ID:

From:

To:

Book



Booking successful! Booking ID: 1
Total price: \$120

OK

Book a Car ✓

Car ID:

From:

To:

Book



You cannot book your own car

OK



This car is already booked for those dates

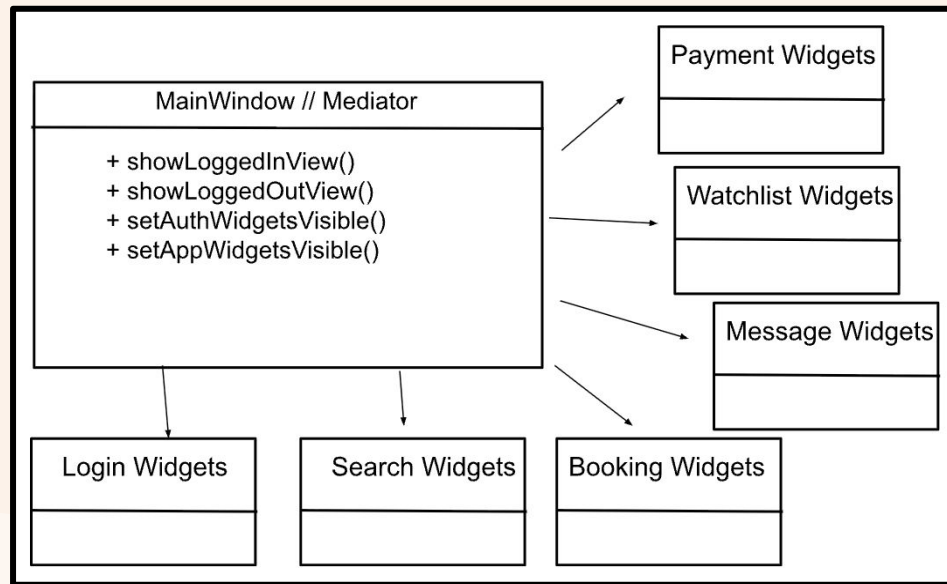
OK

An illustration featuring two hands, one light orange and one brown, exchanging a dark blue car key. A white square box with the number '03' in orange is positioned between them. The background is a light beige color with stylized white clouds, small orange stars, and a landscape at the bottom with orange hills and two trees. The entire scene is framed by orange borders on the left and right sides.

03

Mediator Pattern

UML CLASS DIAGRAM



Implementation:

MainWindow
coordinates
multiple UI
components

```
void MainWindow: showLoggedOutView()
{
    // Mediator: show login tools, hide app
    setAuthWidgetsVisible(true);
    setAppWidgetsVisible(false);
}

void MainWindow::showLoggedInView()
{
    // Mediator: hide login tools, show app tools.
    setAuthWidgetsVisible(false);
    setAppWidgetsVisible(true);
}
```

```
if(authManager.login(email, password, userId)) {
    QMessageBox::information(this, "Success", "Login successful!");

    SessionManager::getInstance()->login(userId);
    showLoggedInView();
}
else {
    QMessageBox::critical(this, "Failed", "Invalid login");
}
```

After login succeeds it
delegates the
responsibility to
showLoggedInView()
which belongs to
MainWindow



Feature: Mediator Pattern for UI Components

Mediator Pattern WHY:

- Reduces direct dependencies between UI components
- Centralizes UI control in one place
- Makes the interface easier to maintain
- Prevents widgets from communicating directly with each other

Mediator Pattern WHAT:

- MainWindow acts as the mediator
- Controls visibility of authentication and application widgets
- Switches between logged-in and logged-out views
- Coordinates login, booking, search, payment, watchlist, and messaging interfaces

DriveShare Login

Email:

Password:

[Forgot Password](#)

[Login](#)

[Register](#)

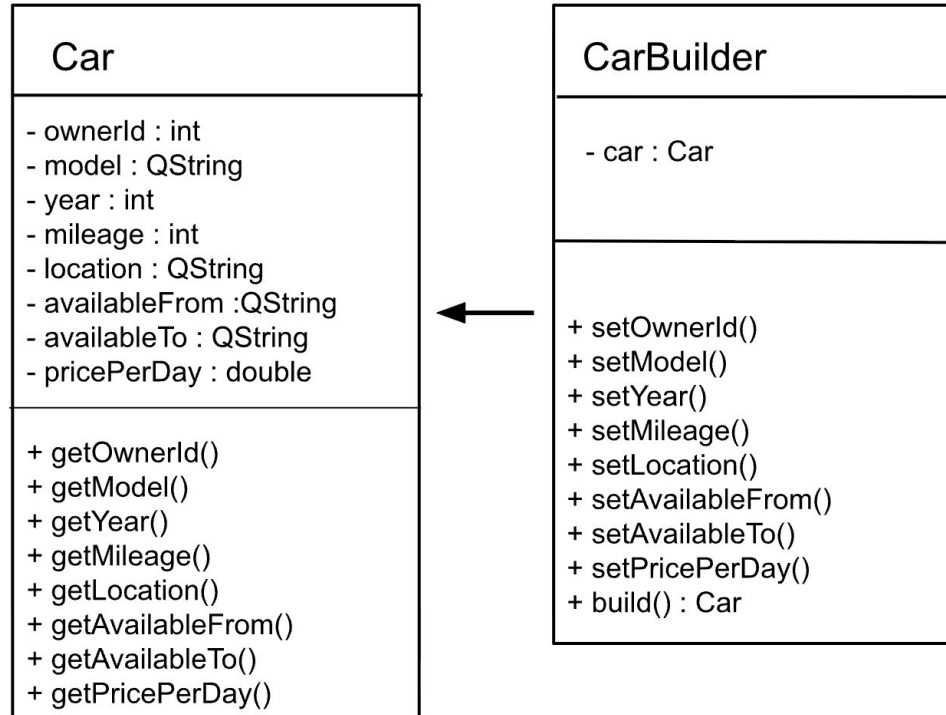
Security Questions and Answers

An illustration featuring two hands, one light orange and one brown, exchanging a dark blue car key. A white square box with the number '04' in orange is positioned between them. The background is a light beige color with stylized white clouds, small orange stars, and a landscape at the bottom with rolling hills and two orange trees. The entire scene is framed by orange borders on the left and right sides.

04

Builder Pattern

UML Class Diagram:



Implementation:

```
✓ class Car
{
public:
    Car();

    int getOwnerId() const;
    QString getModel() const;
    int getYear() const;
    int getMileage() const;
    QString getLocation() const;
    QString getAvailableFrom() const;
    QString getAvailableTo() const;
    double getPricePerDay() const;

private:
    int ownerId;
    QString model;
    int year;
    int mileage;
    QString location;
    QString availableFrom;
    QString availableTo;
    double pricePerDay;

    friend class CarBuilder;
};
```

Class Car: Represents the final product, which is a complete car listing.

```
✓ class CarBuilder
{
public:
    CarBuilder();

    CarBuilder& setOwnerId(int ownerId);
    CarBuilder& setModel(const QString& model);
    CarBuilder& setYear(int year);
    CarBuilder& setMileage(int mileage);
    CarBuilder& setLocation(const QString& location);
    CarBuilder& setAvailableFrom(const QString& availableFrom);
    CarBuilder& setAvailableTo(const QString& availableTo);
    CarBuilder& setPricePerDay(double pricePerDay);

    Car build() const;

private:
    Car car;
```

Class CarBuilder: Responsible for constructing the Car object step-by-step.

Feature: Car Listing



Car WHY:

- Simplifies creation of complex Car objects with many attributes
- Avoids long and confusing constructors with many parameters
- Allows flexible object creation where only some fields may be set
- Improves readability and maintainability of code by building step-by-step

Car WHAT:

- Constructs a Car object using a step-by-step process
- Sets optional and required attributes such as: model, year, mileage, location, and price
- Returns a fully built Car object using the build() method
- Separates object construction (CarBuilder) from the final object representation (Car)

Feature: Listing and Searching:

- **Listing:** Car owners can list their vehicles for short-term rental, providing details such as car model, year, mileage, availability calendar, pick up location and rental pricing.
- **Search:** Renters can search for available cars based on location and date.

List a Car 

Model:

Year:

Mileage:

Location:

From:

To:

Price/day:

Add Car



Car listed successfully

OK

Search Cars 

Location:

From:

To:

Search



Car ID: 2
Model: Ford
Year: 2013
Mileage: 100000
Location: Dearborn
Available: 2026-06-20 to 2026-06-25
Price/day: \$50

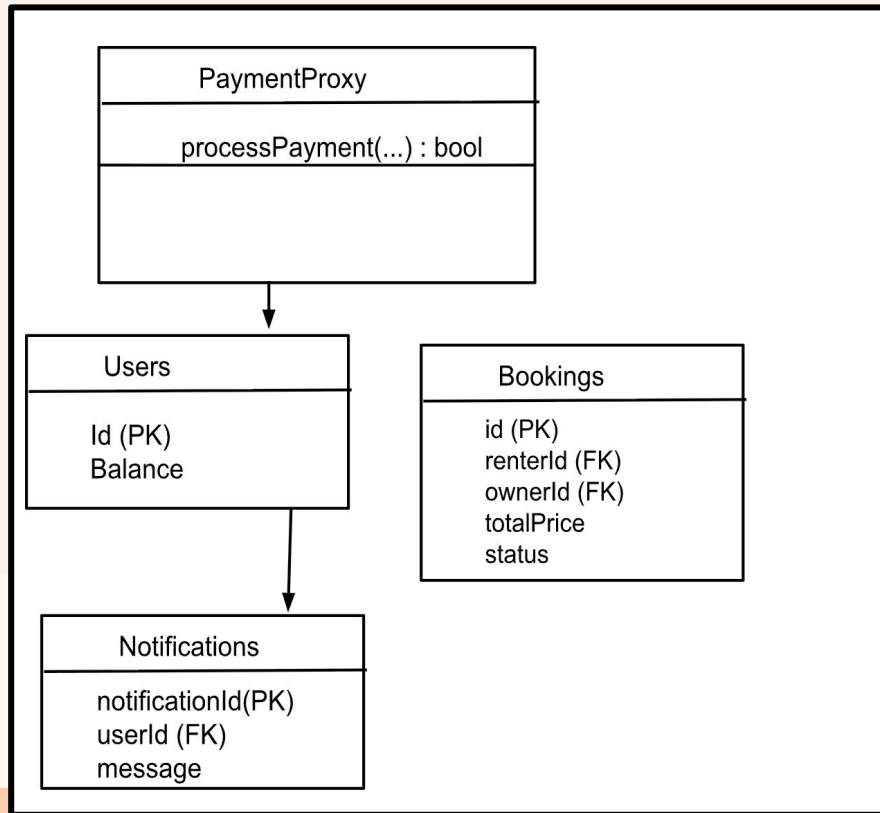
OK

An illustration featuring two hands, one light orange and one brown, exchanging a dark blue car key. A white square box with the number '05' in orange is positioned between them. The background is a light beige landscape with stylized trees, clouds, and small orange stars.

05

Proxy Pattern

UML CLASS DIAGRAM:

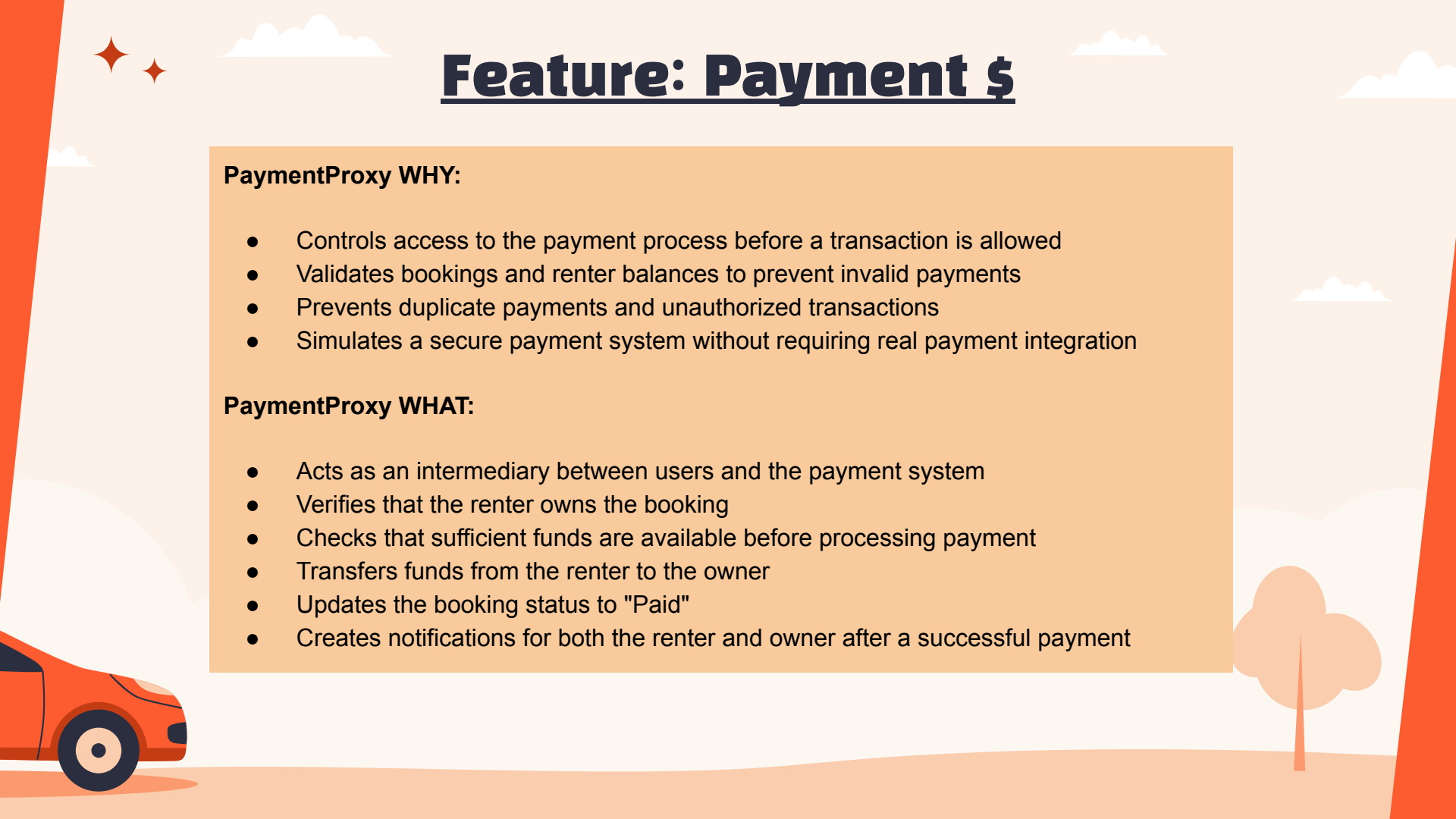


Implementation:

PaymentProxy acts as an intermediary between the user and the payment system. All payment requests must go through this class.

```
1 #ifndef PAYMENTPROXY_H
2 #define PAYMENTPROXY_H
3
4 #include <QString>
5
6 // Proxy: controls access to the fake payment process.
7 class PaymentProxy
8 {
9 public:
10     // Moves fake money from renter to owner and updates booking status.
11     bool processPayment(int bookingId, int renterId, double &amountPaid, QString &errorMessage);
12 };
13
14 #endif // PAYMENTPROXY_H
15
```





Feature: Payment \$

PaymentProxy WHY:

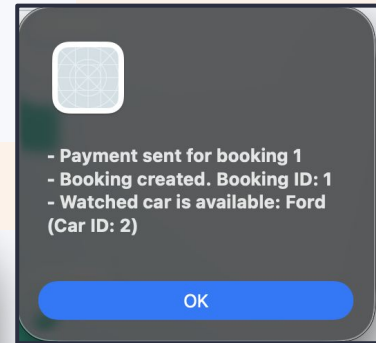
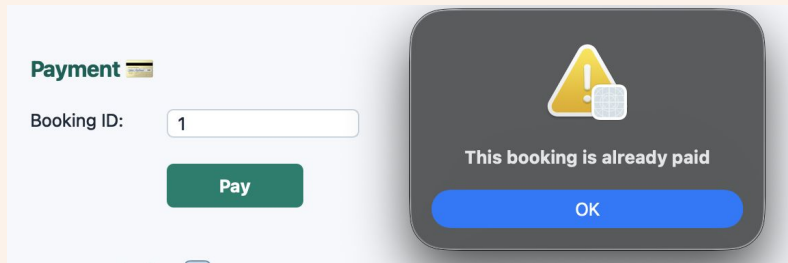
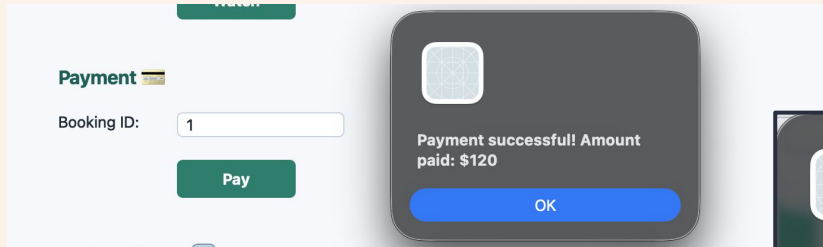
- Controls access to the payment process before a transaction is allowed
- Validates bookings and renter balances to prevent invalid payments
- Prevents duplicate payments and unauthorized transactions
- Simulates a secure payment system without requiring real payment integration

PaymentProxy WHAT:

- Acts as an intermediary between users and the payment system
- Verifies that the renter owns the booking
- Checks that sufficient funds are available before processing payment
- Transfers funds from the renter to the owner
- Updates the booking status to "Paid"
- Creates notifications for both the renter and owner after a successful payment

Feature: Payments

- **Successful Payment:** User can make a payment using their booking ID. User is prompted with success message
- **Double Payment:** User cannot pay twice accidentally. System will notify users that it's already paid.
- **Payment Notifications:** User will receive notification that payment is sent.

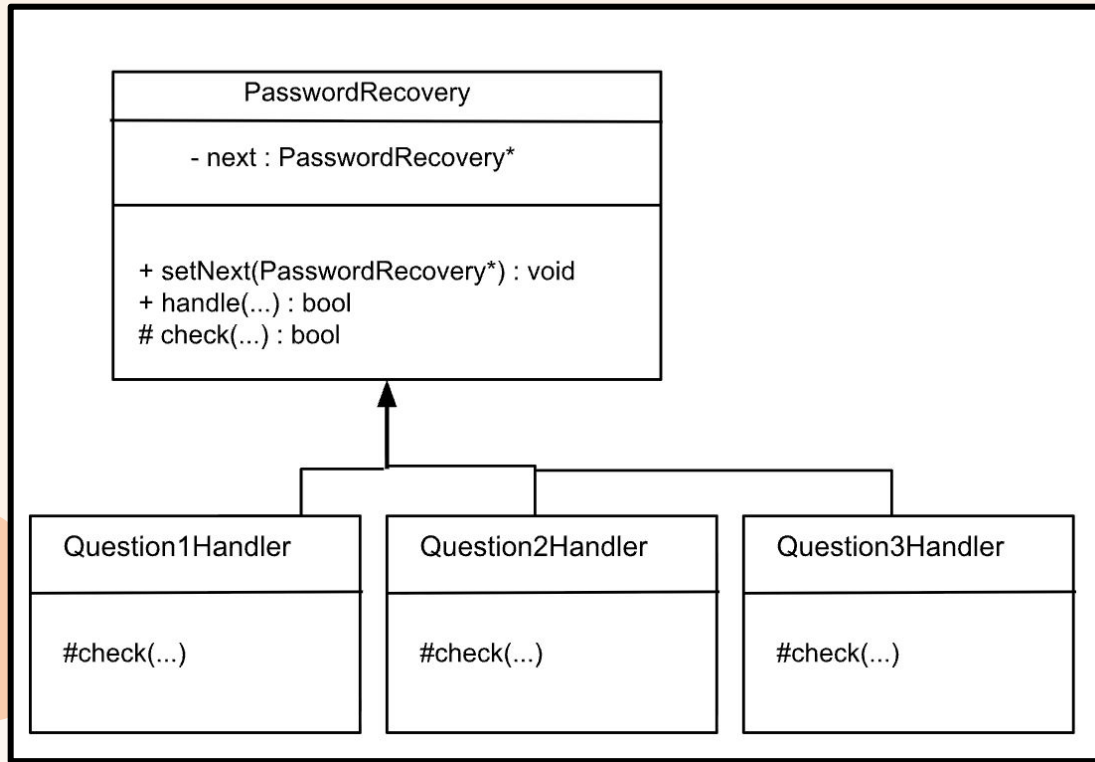




06

COR PATTERN

UML CLASS DIAGRAM:



Implementation:

Stores the address of the next handler in the chain

```
// Chain of Responsibility: each handler checks one security answer.
class PasswordRecovery
{
public:
    PasswordRecovery();
    virtual ~PasswordRecovery();

    // Connects this handler to the next question handler.
    void setNext(PasswordRecovery *nextHandler);

    // Runs this check, then passes to the next handler if correct.
    bool handle(const QString &correctAnswer1, const QString &userAnswer1,
               const QString &correctAnswer2, const QString &userAnswer2,
               const QString &correctAnswer3, const QString &userAnswer3);

protected:
    // Each child class checks one answer.
    virtual bool check(const QString &correctAnswer1, const QString &userAnswer1,
                      const QString &correctAnswer2, const QString &userAnswer2,
                      const QString &correctAnswer3, const QString &userAnswer3) = 0;

private:
    PasswordRecovery *next;
};
```

```
bool PasswordRecovery::handle(const QString &correctAnswer1, const QString &userAnswer1,
                              const QString &correctAnswer2, const QString &userAnswer2,
                              const QString &correctAnswer3, const QString &userAnswer3)
{
    // If this answer is wrong, stop the recovery process.
    if (!check(correctAnswer1, userAnswer1,
               correctAnswer2, userAnswer2,
               correctAnswer3, userAnswer3)) {
        return false;
    }

    // If this answer is right, continue to the next handler.
    if (next != nullptr) {
        return next->handle(correctAnswer1, userAnswer1,
                           correctAnswer2, userAnswer2,
                           correctAnswer3, userAnswer3);
    }

    return true;
}
```

Passes the request to the next handler



Feature: Password Recovery

1. The system allows users to reset their password:
 - 1.1. Click on "Forgot Password".
 - 1.2. System will give you a message.
 - 1.3. Enter the 3 security questions/answers.
 - 1.4. Then click on forgot password again
 - 1.5. If questions/answers are correct, system will reset password
 - 1.6. System will display message

DriveShare Login

Email:

Password:

Forgot Password

Login

Register

Security Questions and Answers

What is my fav color?

Blue

What is my fav food?

Cereal

What is my fav hairstyle?

Braid



Enter your answers and new password, then click Forgot Password again.

OK



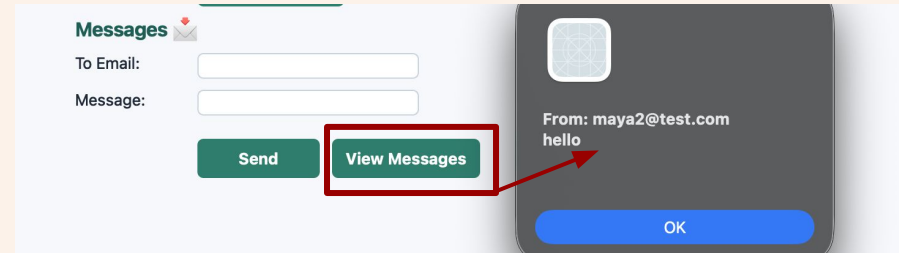
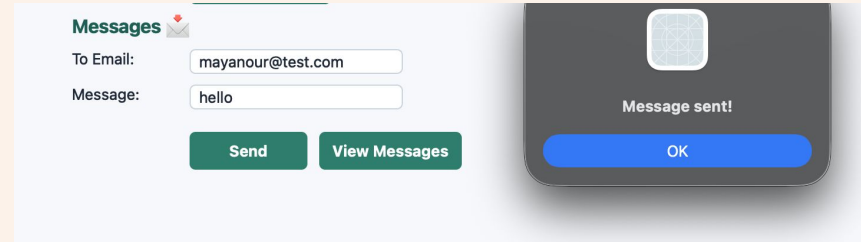
Password reset successful!

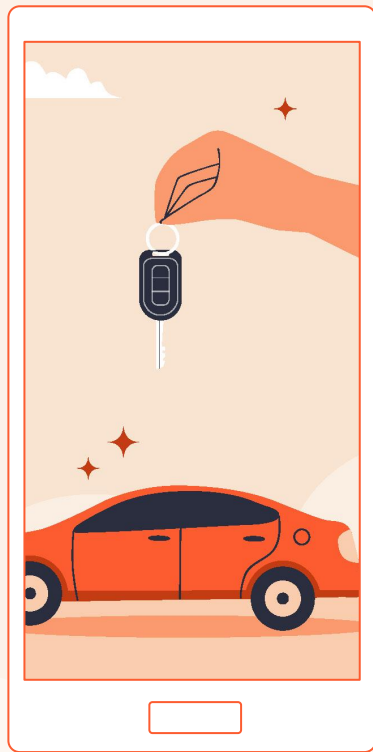
OK

Feature: Message and Communication

Another feature of DriveShare is messaging between renters and owners:

- Accounts can send messages to each other using the registered email address.
- To see messages, click "View Messages"
- If the user has not been sent any messages, the system will display "No messages yet".





Thank you.

